gpxil: Genome Pixel Image format

Version 0.3 (draft), October 2026

Copyright © 2026 Chronomic Medicine Group, Department of Genomic Medicine, University of Cambridge. Released under the MIT licence (see `LICENSE`). Also available as a PDF, `gpxil-spec.pdf`.

gpxil stores a genome as pictures: one pixel per base, one image per chromosome and per kind of information, in an ordinary multi-page TIFF file. A gpxil file opens in any TIFF viewer as a picture of the genome, can be printed, and can be read by image-analysis software as plain arrays, while remaining a lossless representation of the reference sequence and of one person's genotypes.

This document is the format specification. docs/rationale.md explains the design choices and the measurements behind them; gpxiltools is the reference implementation.

Contents

1. Conventions
2. Overview
3. Data model
4. Layout
5. Planes
6. Encoding VCF calls
7. Sidecar
8. GPXIL-TIFF container
9. Metadata
10. Conformance
11. Interpretation (informative)
12. Rendering and printing (informative)
13. Privacy and security considerations
14. Versioning and future work
15. References

- Appendix A: code tables
- Appendix B: worked example

---

1. Conventions

The key words MUST, MUST NOT, REQUIRED, SHOULD, SHOULD NOT, RECOMMENDED, MAY and OPTIONAL are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

Sections marked *informative* explain or advise; everything else is normative.

Numbers written `0x..` are hexadecimal. Bit 0 is the least significant bit. A *nibble* is 4 bits: the *high nibble* of a byte `b` is `b >> 4`, the *low nibble* is `b & 0x0F`.

Term	Meaning
contig	A reference sequence: a chromosome, mitochondrion, scaffold or other named sequence.
offset	A 0-based position on a contig. Offset `i` is VCF `POS = i + 1` and BED start `i`.
plane	One per-base array over a contig, such as the reference sequence or the genotype.
page	One TIFF image (IFD). Each page holds one plane of one contig.
haplotype	One of the two copies of each chromosome in a diploid genome, numbered 1 and 2.
state	What a haplotype carries at a reference base: the reference base, another base, nothing, etc.
sidecar	Per-contig table for information that does not fit in a pixel (inserted sequences, structural variants).
writer / reader	Software that creates / interprets gpxil files.

2. Overview

The idea goes back to Albrecht-Buehler's genome pixel images (GPxI, 2009), in which each base of a sequence was drawn as one pixel, A black, C dark grey, T light grey and G white, wrapping into rows like text

so that repeats line up into visible patterns. gpxil keeps the one-pixel-per-base picture and makes it a file format:

- **Codes, not colours.** Pixels hold integer codes for bases and genotype states. Colours are applied when the image is shown or printed, from a palette; the same file can be rendered in sRGB for a screen or in CMYK for a press without any colour conversion of the data.
- **Planes.** Each kind of information is a separate image of the same shape: the reference sequence, the genotype of each base, flags, quality. Every plane is aligned to the reference, so the same pixel is the same genomic position in every plane and in every person's file.
- **An ordinary container.** A gpxil file is a valid multi-page TIFF. Pages are 8-bit palette images where that makes sense, so standard viewers show a coloured picture of the genome, and image libraries read the planes as arrays.
- **Random access.** Pages are stored in strips of whole rows. Any genomic interval is read by decoding only the strips that contain it.
- **Lossless.** The reference sequence (including soft-masking) and both haplotypes of the sample (substitutions, insertions, deletions, phase, haploid regions, no-calls) can be reconstructed exactly.

A gpxil file holds the reference and at most one sample, over whole chromosomes or over windows of them: a gene and its surroundings, say, in a file of a few kilobytes that keeps the chromosome's positions (section 3.1). It is not a replacement for VCF, CRAM or gVCF in analysis pipelines; it keeps the haplotype content of a call set, not its record syntax or annotations (section 6.9).

3. Data model

A gpxil file consists of **metadata** (section 9) and an ordered list of **contigs**. Each contig has:

- a **name**, unique within the file;
- a **length** `L` ≥ 1, the number of bases;
- a **layout width** `W` ≥ 1 (section 4);
- one or more **planes**, each a sequence of exactly `L` values of a fixed data type, one value per offset `0 .. L-1`;
- an optional **sidecar** (section 7);

- if it holds a stretch of a longer sequence rather than all of it, the **window** it is: which sequence, and where in it (section 3.1).

The standard planes are:

Plane	Type	Content	Section
`ref`	uint8	reference base (IUPAC code) and soft-mask bit	5.1
`gt`	uint8	sample genotype: state of haplotype 1 and haplotype 2	5.2
`flags`	uint8	per-base flags: phased, insertion follows, filtered, called	5.3
`gq`	uint8	genotype quality	5.4
`dp`	uint8	read depth	5.4
`ps`	uint32	phase set	5.5
`x-…`	see 5.6	extension planes	5.6

Rules:

1. A contig SHOULD have a `ref` plane. A contig without one MUST carry the `md5` and/or `refget` digest of its reference sequence in its metadata entry (section 9.3), or, for a window, those of its parent (section 3.1), so that readers can obtain the sequence elsewhere.
2. If a contig has a `gt` plane it MUST also have a `flags` plane.
3. `gq`, `dp` and `ps` planes MAY be present only when a `gt` plane is.
4. All contigs of a file that carry a `gt` plane describe the same sample, the one in the `sample` metadata (section 9.4).
5. A plane name used on several contigs MUST have the same data type on all of them.

3.1 Windows

A **window** is a contig that holds a stretch of a longer sequence, its **parent**, rather than all of it: a gene and its surroundings, say, out of a person's whole genome. Its metadata entry has a `window` key (section 9.3) that names the parent and gives the window's **start**, the offset in the parent of the window's offset 0. Offset `i` of the window is offset `start + i` of the parent, for `0 ≤ i < L`.

Rules:

1. A window holds what a contig of its whole parent, made from the same inputs by the same rules, would hold at offsets `start .. start + L − 1`: in each of its planes the parent's values at those offsets, and in the sidecar the parent's entries at those offsets (section 7), each at its offset less `start`. So calls are converted as for the whole parent (section 6): a call that crosses an end of the window writes the bases inside it as it would in the whole parent, phase-set identifiers take the values they take on the whole parent (section 5.5), and a window has a `gt` plane when its whole parent would have one (section 6.1), even if no call falls in it, and none when its parent would have none. Like any contig, a window MAY leave planes out (as a copy of a reference's bases only does). The one thing a window holds that its parent does not hold at the same base is a copy of a COMPLEX entry at its first base (below).
2. `start + L` MUST NOT exceed the parent's length, and a window holds less than all of its parent: a contig of all of a sequence is the sequence, without a `window` key. The parent is a sequence, never another window: a window cut from a window names that window's parent, and its own start in it.
3. The window's name MUST differ from the parent's name and from the parent's other names, so that a reader asked for the parent never takes the window for it; writers SHOULD give the parent's other names in the `window` key (section 9.3), so that this can be checked. A reader that matches names loosely (ignoring case, or a `chr` prefix) MUST match only whole contigs so: a window is found by its name as written. The name SHOULD be the parent's name followed by the window's first and last positions in the parent, 1-based, as `<parent>:<first>-<last>` (`chr17:7561779-7787546`): the way most tools write the region it holds.
4. The window's `md5` and `refget` (section 9.3) are digests of its own sequence. The parent's digests and length SHOULD also be given, in the `window` key, so that the parent can be found and checked; a window without a `ref` plane needs its own digests or its parent's (section 3, rule 1), and the parent's identify it as well, with its start and length (a refget server, for one, serves a stretch of a sequence).
5. A window without a `ref` plane takes its bases from its parent: a reader given the reference finds the parent (by name, other names or digests), checks it against what the `window` key and the window's

own digests say, and reads offsets `start .. start + L − 1` of it.
6. A file MAY hold several windows of one parent, which MAY overlap, and MAY also hold the parent itself.
7. Readers SHOULD give positions on a window in its parent's coordinates (offset `i` is position `start + i + 1`) and SHOULD accept regions in them, reading a region of the parent from a window that holds all of it.

In everything else a window is a contig like any other, with its own layout (section 4: its first base is row 0, column 0) and its own metadata entry. A sidecar entry belongs to its base, so an insertion after the base before the window is not in the window, nor is a COMPLEX allele that starts before it, though the allele's `END` may reach into it. If the window's first base is COMPLEX on a haplotype and the parent has no entry there (a run of COMPLEX bases that started before the window), the window holds, at offset 0, a copy of the parent's nearest entry before it on that haplotype (the entry of the allele the base belongs to), so that every run starts with an entry (section 7).

A writer need not convert every call of the parent to make a window (informative). Calls are applied in file order (section 6.4), and a base depends only on the calls whose spans (section 6.2) cover it, and, through section 6.5, on the calls whose spans share a base with those, and so on. So the calls whose spans share a base with the window, and those whose spans share a base with them, and so on, converted alone, in file order and from the initial states of section 6.1, over the stretch their spans cover, give the window's values; whether the window has a `gt` plane, and the values of phase-set identifiers (section 5.5), come from the whole parent. gpxiltools makes windows both ways, by converting calls (`gpxil encode --region`) and by cutting them out of a file (`gpxil subset`), and checks that the two agree.

4. Layout

Planes are stored as 2-D raster images. With layout width `W`, offset `i` is stored at pixel

```
row = i div W      (counting from the top, starting at 0)
col = i mod W      (counting from the left, starting at 0)
```

so bases run left to right and rows run top to bottom, like lines of text. An image has `rows = ceil(L / W)` rows and `W` columns. Pixels at `row * W + col ≥ L` are **padding**: writers MUST set every padding pixel of every plane to 0 and readers MUST ignore padding.

`W` is chosen by the writer and recorded per contig (section 9.3). Files meant to be compared pixel by pixel SHOULD use the same `W`; because the layout is a reshape of a 1-D array, a reader can always re-wrap a plane to any other width without loss. gpxiltools uses `W = 8192` by default.

Other arrangements, such as page-sized rows for printing (section 12.4), a width equal to a repeat period to make tandem repeats stand out as vertical stripes, or a Hilbert curve that keeps neighbouring bases neighbouring in two dimensions (section 11.5), are *views* computed from the raster. This version stores only the raster layout.

5. Planes

Appendix A lists every code value. Values marked *reserved* MUST NOT be written; readers SHOULD report them and MUST NOT interpret them as anything other than "unknown".

5.1 `ref`: reference sequence

One byte per base:

Bits	Meaning
0-3	IUPAC nucleotide code as a set over {A, C, G, T}: A = 0x1, C = 0x2, G = 0x4, T = 0x8; ambiguity codes are unions (R = A|G = 0x5, N = 0xF). 0x0 means *no base*.
4 (0x10)	soft-masked: the base was lowercase in the source FASTA (repeat-masked).
5-7	reserved, MUST be 0.

The 4-bit set is the base encoding used for read sequences in BAM, so the code of every IUPAC symbol is the bitwise OR of the bases it stands for (Appendix A.1). A soft-masked base without a base (0x10) is invalid.

When reading FASTA, writers MUST map `A C G T` and the IUPAC ambiguity letters case-insensitively as above, setting the soft-mask bit for lowercase letters; MUST map `U` like `T`; MUST map `-` and `.` to 0x00; and MUST either reject any other character or map it to N (0x0F) and record how many were converted in the contig's `invalid_characters_converted_to_N` (section 9.3). Writers MUST NOT drop characters: offset `i` is always base `i + 1` of the FASTA record. The contig name is the header line up to its first whitespace character (space or tab). Records with no sequence are not contigs (section 3) and are skipped.

5.2 `gt`: genotype

One byte per base: the **high nibble** is the state of haplotype 1, the **low nibble** the state of haplotype 2. States are relative to the reference base at the same offset:

State	Name	Meaning
0x0	REF	the haplotype carries the reference base
0x1	A	the haplotype carries A (a base other than the reference)
0x2	C	... C
0x4	G	... G
0x8	T	... T
0xA	ABSENT	the haplotype does not exist here (one copy only: haploid region)
0xC	COMPLEX	the haplotype carries something not expressible per base; see the sidecar
0xD	DEL	the reference base is deleted on this haplotype
0xF	UNKNOWN	the state is not known (no call)
others	—	reserved

The single-bit states use the same bits as section 5.1, so the base on a haplotype is the state itself when the state is a base, and the reference base when the state is REF. The letters of the special states are mnemonics: **A**bsent, **C**omplex, **D**eleted, and 0xF ("all bits") for unknown, as N is 0xF in `ref`.

Rules:

1. A base state MUST NOT equal the reference base; a haplotype carrying the reference base is REF (0x0). (Where the reference base is ambiguous or N, any base state is allowed.)
2. For phased calls (section 5.3, PHASED) haplotype 1 and haplotype 2 are the first and second alleles of the phased genotype. For unphased calls, haplotype 1 holds the first allele written in the genotype and haplotype 2 the second (but see section 6.5 for `*` alleles and for calls that overlap others); the order then carries no biological meaning.
3. A haploid call sets haplotype 2 to ABSENT. Polyploid calls cannot be represented (section 14).
4. Insertions are not states: they happen *between* bases and are flagged in the `flags` plane (INS1, INS2) on the base after which they occur, with the inserted sequence in the sidecar.

Examples: `0x00` homozygous reference; `0x04` heterozygous with G on haplotype 2 (`0|1`, ALT G); `0x44` homozygous G; `0x48` heterozygous with G and T (`1|2`); `0x0D` a base deleted on haplotype 2; `0xDD` deleted on both; `0x1A` haploid A; `0xFF` no call.

5.3 `flags`

One byte per base, a bit field:

Bit	Name	Meaning
0x01	PHASED	the call covering this base was phased (`|`), so haplotype order is meaningful
0x02	INS1	an insertion follows this base on haplotype 1 (sequence in the sidecar)
0x04	INS2	an insertion follows this base on haplotype 2
0x08	FILTERED	a call covering this base did not pass its filters (VCF FILTER not PASS or `.`)
0x10	VARIANT_SITE	the base is covered by a call for this sample (any genotype)
0x20-0x80	—	reserved, MUST be 0

VARIANT_SITE distinguishes "called reference" (state REF, flag set) from "reference by default" (state REF, flag clear) in files made from a VCF that lists only variant sites.

5.4 `gq` and `dp`

One byte per base: genotype quality (`gq`) or read depth (`dp`) of the call covering the base. Values above 254 MUST be stored as 254. The value 255 means *not available* and MUST be used for bases not covered by a call with that field.

5.5 `ps`

One unsigned 32-bit integer per base: the phase set of the phased call covering the base (section 6.5), or 0 if none; a base whose PHASED flag is clear MUST have 0. Where the VCF phase set (`FORMAT/PS`) is an integer from 1 to 2^32−1 it MUST be stored as is. An identifier written in the digits 0-9 is an integer and is compared as one (`077` is 77, `00` is 0); any other is compared as written. Other phase-set identifiers (0, an integer above 2^32−1, or any that isn't an integer) MUST be replaced by values that no integer identifier of those calls takes, and the writer MUST record the mapping in the contig's `phase_set_ids` metadata. The calls are the contig's phased calls in the VCF: calls of two alleles, written phased, with a `PS` other than `.` or empty, that are not skipped (sections 6.2 and 6.5). The values MUST be taken downwards from 2^32−1, skipping the integers that those calls' identifiers are, in order of first appearance among those calls, counting every identifier, including one that ends up on no base because other calls took all of its bases, which then has no entry in `phase_set_ids`. For a window (section 3.1) the calls are those of its whole parent, so a window's values are its parent's, and the same in every window of the parent. When a contig has phased calls but no `ps` plane, all its phased calls belong to one phase set (as for phasing from a diploid assembly or a trio).

5.6 Extension planes

Writers MAY add planes named `x-` followed by letters, digits, `-` or `_`, for example `x-methylation` or `x-coverage`. An extension plane MUST have one of the types uint8, uint16, uint32, int8, int16, int32 or float32, and MUST be described in the `planes` metadata (section 9.2). Readers MUST ignore extension planes they do not understand.

6. Encoding VCF calls

This section fixes how one sample's calls in a VCF (version 4.2 or later) become `gt`, `flags`, `gq`, `dp`, `ps` and sidecar data, so that different writers produce the same planes from the same input. Calls are matched to contigs by name (section 6.8); calls on contigs the reference does not have are skipped, and writers SHOULD list those contigs in the metadata. For a window (section 3.1) the contig here is its whole parent, and the window holds the result over its stretch.

6.1 Initial state

Before any call is applied, in this order:

1. every base of both haplotypes is REF;
2. if the writer is given *callable regions* (for example a BED file of confidently called intervals), both haplotypes are UNKNOWN outside them; a contig that the regions do not mention at all is UNKNOWN throughout;
3. if the writer is given *haploid regions* (for example chromosome X outside the pseudoautosomal regions in a male), haplotype 2 is ABSENT inside them, whether or not they are callable.

These initial states are not *asserted* (section 6.4). The metadata MUST say which callable and haploid regions were applied (section 9.4).

A contig on which the VCF has no records and which no callable interval overlaps (because no callable regions were given, or none lies on the contig) carries no information about the sample: the default of step 1 would claim that the person matches the reference there. Writers MUST NOT give such a contig a `gt` plane; it is stored as reference only, or, by a writer told not to store the reference, left out and SHOULD be listed in the metadata. (Without a callable-regions file, a VCF that lists only variant sites is taken to mean REF wherever a contig *with* calls has none, the convention of `bcftools consensus`.)

BED files are read with columns separated by tabs or spaces; `track`, `browser` and `#` lines are ignored.

6.2 Allele decomposition

A call has a REF allele of length `m` starting at offset `p = POS − 1`; it covers the *span* `[p, p + m)`, extended to `[p, END)` for records with a symbolic allele and for reference blocks (section 6.3), `END` being `INFO/END` (a 1-based inclusive position) or, for a record with a `<DEL>` allele and no `INFO/END`, the largest `END` that its `<DEL>` alleles' `INFO/SVLEN` values give (`POS + |SVLEN|`, section 6.3); an `END` that is not after the span's end leaves it as it is. A record whose `POS` lies outside the contig, or whose REF allele is empty or runs past its end, MUST be skipped and counted (`skipped_out_of_range`); a span that `END` extends past the end of the contig is cut at the end and counted (`clipped_to_contig_end`). REF and sequence ALT alleles are read case-insensitively.

Each haplotype's allele is turned into per-base states over the span as follows:

- `0` (the REF allele): REF over the span.
- `.` (missing): UNKNOWN over the span.
- `*` (spanning deletion): nothing is written; the deletion is described by the record that starts it (see also section 6.5).
- an allele that is not a number written in the digits 0-9 (so `-1` and `+1` are not), or whose number is larger than the number of ALT alleles: nothing is written; the writer counts it (`bad_allele_index`).
- a sequence allele `ALT`:

  1. Trim the longest common suffix of REF and ALT, except that when the lengths differ the trim MUST stop before consuming the whole shorter allele, so the anchor base remains.
  2. Then trim the longest common prefix of what remains; let `k` be its length.
  3. The remaining cores `r` (from REF) and `a` (from ALT) start at offset `p + k`. For `j < min(|r|, |a|)`, where `r[j] ≠ a[j]` the state at `p + k + j` is the base `a[j]`, with `U` read as `T`, or UNKNOWN if `a[j]` is not A, C, G, T or U. If that base is the reference base at the offset (possible only when the record's REF disagrees with the reference), the state is REF, and it is written as REF (section 6.4).

4. If `a` is longer, the surplus `a[|r|:]` is an insertion after offset `p + k + |r| − 1` (section 7 gives how its bases are stored).
5. If `r` is longer, offsets `p + k + |a| .. p + k + |r| − 1` are DEL.
6. All other offsets of the span are REF.

Because step 1 never consumes all of REF when ALT is longer, the anchor of an insertion is always a base of the record. An insertion before the first base of a contig (VCF `POS=1`, `REF=A`, `ALT=TA`) therefore becomes the substitution A→T at offset 0 followed by the insertion of `A` after it, which spells the same haplotype.

Trimming the suffix first places indels as far left as the record allows, the same convention as VCF left-normalisation, which keeps them consistent with neighbouring calls inside repeats. Any alignment that yields the same haplotype sequence would be equally correct; fixing one makes writers agree.

6.3 Symbolic alleles and reference blocks

An allele is *symbolic* if it is written in angle brackets (`<DEL>`), is a breakend (contains `[` or `]`, as in `G]chr17:198982]`), or is a single breakend (starts or ends with `.`, as in `G.` or `.G`).

- `<NON_REF>`, `<*>` and `<X>` stand for "any other allele". A record whose ALT alleles are only these, or that has no ALT allele, is a reference block: with `INFO/END` it covers `[p, END)`, and its `0` alleles assert REF over that span. A haplotype whose allele *is* one of them is UNKNOWN over the span.
- `<DEL>` and its subtypes (`<DEL:ME>` …): POS is the padding base before the deletion, by VCF convention, so offset `p` is REF and offsets `p+1 .. END−1` are DEL. Without `INFO/END`, the allele's own `INFO/SVLEN` value (one per ALT allele) gives `END = POS + |SVLEN|`. With neither, or with an END that is not after POS, the allele is treated as below.
- Any other symbolic allele, including breakends and single breakends: the haplotype is COMPLEX at offset `p`, and a COMPLEX sidecar entry records the allele (sections 6.6 and 7). This is all the allele asserts: what the haplotype carries over the rest of the span (an inverted or duplicated sequence, say) cannot be written per base, so those bases keep their initial state unless other calls set

them, and the entry's `END` tells readers how far the allele reaches. `SVLEN` of these alleles (the length of an insertion or duplication, say) does not change the span.

Every other allele of a record with a symbolic allele (`0`, `.`, a sequence allele) covers the record's whole span, as in section 6.2.

6.4 Precedence

Calls are applied in file order. Each haplotype base remembers whether a call has *asserted* it. The states A, C, G, T, DEL and COMPLEX are *strong*: they say what the haplotype carries. REF, UNKNOWN and ABSENT are *weak*. When a call writes a state to a base:

1. A strong state always replaces the current state. If the base was already asserted with a different strong state, this is a *conflict*: the later call wins and the writer MUST count it (section 9.4).
2. A weak state replaces the current state only if no call has asserted the base yet. ABSENT written by a haploid call over a base already asserted with a strong state is also a conflict (the strong state stays).
3. The base is now asserted.

So a record whose REF allele overlaps another record's variant never undoes that variant, whatever the record order, and overlapping records that use `*` combine correctly. Split multi-allelic records (two records at one position, each `0/1`) go one to each haplotype when they are unphased (section 6.5). Phased, written `0|1` both, they are applied to haplotype 2 as written, and only different strong states conflict: two substitutions do, but a deletion inside a longer one from the same anchor leaves the longer one, and no conflict.

Insertions are added at their anchor base on their haplotype; two different insertions at the same base on the same haplotype are a conflict (last wins), different meaning that the sidecar stores different bases (section 7: inserted `R` and `Y` are both stored as `N`, so they don't conflict). The same holds for two different COMPLEX alleles at one base and haplotype: alleles whose REF, ALT or END differ. A COMPLEX state that a later call replaces takes its sidecar entry with it.

A call never meets the initial states of section 6.1 as a conflict: they are not asserted. In particular a diploid call inside a haploid region writes haplotype 2 like any other base, because the VCF is the more specific statement; writers SHOULD count such bases (`ploidy_mismatches`).

6.5 Ploidy, phase and flags

- The genotype is `FORMAT/GT`. A leading phase indicator (VCF 4.4, as in `|1` or `/0|1`) is dropped before the alleles are read.
- A missing genotype (`.`, empty, or no `GT` field) leaves the number of alleles unknown: both haplotypes are UNKNOWN over the span, except that a base whose haplotype 2 is ABSENT keeps it, without this call asserting it. Being weak, this never erases another call's variant. Writers SHOULD count such records (`missing_genotypes`). With a leading phase indicator (`|.`) the genotype has one allele, so it is a haploid call whose allele is missing, not a missing genotype.
- A genotype with one allele (`0`, `1`, ...) is haploid: haplotype 2 is ABSENT over the span. Genotypes with more than two alleles MUST be skipped and counted (`polyploid_skipped`).
- A diploid genotype whose separator is `|` is phased: PHASED is set over the span. Haploid calls are not marked PHASED.
- In an *unphased* diploid genotype with exactly one `*` allele, the order of the alleles carries no meaning, so the `*` allele belongs to the haplotype that carries the deletion it refers to: if exactly one haplotype already has an asserted DEL within the span, and it is not the one the written order gives `*`, the two alleles are swapped (counted as `spanning_deletions_reassigned`). Phased genotypes are applied as written.
- In an unphased diploid genotype whose two alleles differ and neither is `*`, the order of the alleles carries no meaning either. But calls that overlap are alternatives at one place: the ALT alleles of two heterozygous calls that overlap cannot both be on one haplotype, and written `0/1`, both would go to haplotype 2 and run together. So each allele goes to a haplotype it *fits*. An ALT allele *occupies* the offsets of its cores (section 6.2, step 3) and, when one core is empty (a plain insertion or deletion), the offset before them,

its anchor; a `<DEL>` allele occupies offsets `p .. END - 1`, any other symbolic allele offset `p` (section 6.3). An allele fits a haplotype unless an earlier call put an ALT allele there that occupies an offset this one occupies, or put this same allele on the other haplotype. The same allele again (the same cores at the same offset, `U` read as `T`; for a symbolic allele, the same `p`, ALT and END), as when a record is written twice, fits a haplotype that already has it, so it goes there whichever way its genotype is written. The REF allele `0`, a missing allele `.`, an allele that writes nothing (section 6.2) and the "any other allele" placeholders (section 6.3) occupy nothing and fit any haplotype. If an allele does not fit the haplotype that the written order gives it, and the two alleles swapped both fit, they are swapped (counted as `overlapping_calls_reassigned`); otherwise they are applied as written, and section 6.4 decides. Each call is placed against the calls applied before it, phased or not, in file order, and after the rule for `*` alleles above; phased genotypes are applied as written. For example:

```
#CHROM POS REF ALT GT
chr1   100 TTC T   0/1
chr1   102 CT  C   0/1
```

The first record's ALT allele occupies offsets 99-101 of haplotype 2: its anchor, and the two bases it deletes. The second's occupies offsets 101-102, its anchor (a base the first deletes) and the base it deletes, so it does not fit haplotype 2; it goes to haplotype 1, and its REF allele to haplotype 2. Haplotype 2 then carries the first deletion (offsets 100-101) and haplotype 1 the second (offset 102), as the records mean, not one deletion of three bases.

Because an allele occupies what it changes and no more, split multi-allelic records (two records at one position, each `0/1`) go one to each haplotype, whatever their alleles, unless they change different bases of a longer REF (REF `CG`, ALT `TG` and `CA`: two substitutions, one at each base), which occupy no offset in common and are applied as written. And an allele padded to the REF of a multi-allelic record occupies only what it changes: in `CTTTA` to `CTA,C`

with genotype `1/2`, `CTA` occupies the offsets of `CTT` to `C`, so a heterozygous call at the record's last base, which only `C` deletes, goes to the haplotype of `CTA`.

- Flags accumulate (bitwise OR) over all calls covering a base. VARIANT_SITE is set over the span of every call; FILTERED over the span of every call whose FILTER is neither `PASS` nor `.`.

- `gq` and `dp` take `FORMAT/GQ` and `FORMAT/DP` (capped at 254, negative values stored as 0); `ps` takes `FORMAT/PS` of phased diploid calls (section 5.5). Each is written over the span of the call. Where several calls with the field cover a base, one that changed the base (wrote a strong state to it on either haplotype, or an insertion after it) takes precedence over those that did not; among equals the later one wins. Bases no call with the field covers keep the missing value (255 for `gq` and `dp`, 0 for `ps`).

6.6 COMPLEX

A haplotype is COMPLEX where its allele cannot be expressed per base: the symbolic alleles of section 6.3 other than deletions. The state is written at the record's first base only, together with a COMPLEX sidecar entry for that offset and haplotype, so every run of COMPLEX bases starts with an entry (section 7). Readers treat COMPLEX bases as unknown sequence and find what is known about the allele in the entry.

6.7 Reference checks

Writers MUST check each REF allele against the reference and MUST count mismatches in the metadata (`ref_mismatches`). A REF base matches when it is an IUPAC letter (or `U`) and its set of bases and the reference base's set intersect: N matches any base but not a gap (`-`), R (A or G) matches A or G, and `-`, `.`, `*` or any other character matches nothing. Writers MAY reject files with mismatches.

6.8 Contig names

Writers MAY match VCF contig names to reference names that differ only by a `chr` prefix, or by `MT`/`M`/`chrM`, and SHOULD record the alternative names as `aliases` (section 9.3). Contig names MUST meet

section 9.3 (printable ASCII, no whitespace); a writer that cannot store a name MUST fail rather than change it silently.

6.9 What is and is not kept

Kept exactly: the reference sequence and soft-masking; the sequence of both haplotypes (every substitution, deletion and insertion); phasing; haploid calls; no-calls; the FILTER pass/fail status, genotype quality, depth and phase set per base.

Not kept: VCF record boundaries and allele syntax, IDs, QUAL, INFO, other FORMAT fields, other samples, records outside the reference's contigs, and the exact bases of structural alleles (kept only as COMPLEX entries). Converting a gpxil file back to VCF (section 11.4) yields records that are equivalent in haplotype content to the input, not a copy of it.

7. Sidecar

Each contig with a `gt` plane MAY have a sidecar: a table of events that do not fit in pixels. It is UTF-8 text with tab-separated columns:

```
#gpxil-sidecar  0.1
offset  hap kind    data
5217935 1   INS AAAAAGA
5217935 2   INS AAAAAGA
9812201 2   COMPLEX REF=A;ALT=<INV>;END=9813300
```

Column	Content
`offset`	0-based offset of the base the entry belongs to, on the contig (on a window, the window's offset: section 3.1)
`hap`	1 or 2
`kind`	`INS` or `COMPLEX`
`data`	for `INS`: the inserted bases, one or more of uppercase `A C G T N`; for `COMPLEX`: `;`-separated `KEY=value` items, at least `REF` and `ALT`, and `END` (VCF coordinates, so on a window the parent's) when known

Rules:

1. The first line is `#gpxil-sidecar` and the sidecar's version, `0.1` (its own: versions 0.2 and 0.3 of gpxil did not change it), the second the column header; then one entry per line, sorted by offset, then hap, then kind.
2. For every base with INS1 (INS2) set there MUST be exactly one `INS` entry with that offset and hap 1 (2), and every `INS` entry MUST have its flag set.
3. Inserted bases are stored in uppercase with `U` as `T`; characters of the VCF allele other than A, C, G, T, U and N (an IUPAC ambiguity code, say) are stored as `N`, and the writer counts them (`nonacgt_inserted_bases`).
4. Every run of COMPLEX bases on a haplotype MUST start with a `COMPLEX` entry for its first offset and that haplotype, and there MUST NOT be two `COMPLEX` entries for one offset and haplotype. Entries on bases that are not COMPLEX SHOULD NOT be written; readers ignore them.
5. `COMPLEX` data keep the VCF allele as written, including the case of breakend mate contig names.
6. Readers MUST ignore entries of unknown kinds and lines starting with `#`.

8. GPXIL-TIFF container

8.1 File

A gpxil file is a TIFF file (TIFF 6.0, or BigTIFF when larger than 4 GiB) in either byte order. Writers MUST use BigTIFF when the file may exceed 4 GiB and SHOULD use classic TIFF otherwise, because more software reads it. (gpxiltools writes classic TIFF, and starts again as BigTIFF if the data still to be written could take the file past 4 GiB less 64 MiB; a whole human genome with one sample takes under 1 GB.)

The RECOMMENDED file name extension is `.gpxil.tif`, which tells people what the file is while letting ordinary software open it; the media type is `image/tiff`. Readers MUST recognise a gpxil file by its metadata (section 9), not by its name.

8.2 Pages

Every plane of every contig is one page (IFD). Pages appear contig by contig in the order of the `contigs` metadata list; within a contig, in the order `ref`, `gt`, `flags`, `gq`, `dp`, `ps`, then extension planes in lexical order. Page 0 MUST be a data page, the first plane of the first contig, and MUST hold the file metadata (section 8.4). Readers MUST locate pages through the metadata and MUST ignore pages it does not list.

Because consecutive pages often have the same shape and type, libraries that group pages into series (tifffile, imageio) may offer a contig's `ref`, `gt` and `flags` pages as one `(3, rows, W)` stack of codes, or several contigs of equal size as one stack. That is a convenience of those libraries; the metadata remains the way to tell which page is which.

8.3 Tags

Each data page has these tags:

Tag	Value
ImageWidth (256)	`W`
ImageLength (257)	`ceil(L / W)`
BitsPerSample (258)	8 for uint8/int8 planes, 16 or 32 according to type
SamplesPerPixel (277)	1
SampleFormat (339)	1 (unsigned), 2 (signed) or 3 (float) according to type; may be omitted for unsigned
PhotometricInterpretation (262)	3 (palette) for `ref`, `gt` and `flags`; 1 (BlackIsZero) for all others
ColorMap (320)	for palette pages: display colours (section 8.5)
Compression (259)	see below
Predictor (317)	1 (none) or absent for `ref`, `gt` and `flags`; any for extension planes
ImageDescription (270)	page metadata (section 8.4)
PageName (285)	RECOMMENDED: `"<contig> <plane>"` for display
StripOffsets, RowsPerStrip, StripByteCounts or the tile tags	data layout

Tag	Value
65400 (GPXIL_SIDECAR)	on the `gt` page only, when the contig has a sidecar: the sidecar text compressed with zlib (RFC 1950), type UNDEFINED

Tag 65400 lies in the TIFF range reserved for reusable private tags. It is not registered, so tools built on libtiff (ImageMagick, for one) print a warning such as "Unknown field with tag 65400" and otherwise ignore it; the pages still read correctly.

Compression. Readers MUST support no compression (1) and Deflate (8, and the obsolete code 32946). Readers SHOULD support LZW (5), PackBits (32773), Zstandard (50000) and LZMA (34925). Writers SHOULD use Deflate, which every TIFF reader supports; on chromosome 21, Zstandard (level 19) gives a file about 6 % smaller and LZMA about 11 % smaller, at the cost of compatibility with some viewers.

Strips. Writers SHOULD store pages in strips (whole rows), sized so that a strip holds between 64 KiB and 1 MiB of uncompressed data; gpxiltools uses 256 KiB. Then a genomic interval maps to a run of rows and only the strips holding those rows need decoding. Tiles are allowed; readers MUST support both.

8.4 Page metadata

The ImageDescription of every page except page 0 MUST be a JSON object with at least:

```
        {"gpxil": "0.3", "contig": "chr21", "plane": "gt", "length":
46709983}
```

The ImageDescription of page 0 MUST be the file metadata (section 9), which also contains `contig`, `plane` and `length` for page 0 itself. JSON MUST be written as 7-bit ASCII, with any other characters escaped (`\uXXXX`), so that it is also a valid TIFF ASCII field.

8.5 Colour maps

The ColorMap of a palette page is a viewing aid only: it makes the page a recognisable picture in an ordinary image viewer. Readers MUST NOT use it to interpret pixel values. The `ref` colour map SHOULD follow one of the palettes in section 12.2 and its name SHOULD be given in the `palette` metadata key. The colours of the `gt` and `flags` maps are not specified.

9. Metadata

The file metadata is a JSON object (RFC 8259). schema/gpxil-0.3.schema.json is a JSON Schema for it (0.2 and 0.1 for files of those versions). Readers MUST ignore keys they do not know.

9.1 Top level

Key	Required	Content
`gpxil`	yes	format version, `"0.3"`: the version whose rules the file follows (below)
`layout`	yes	`{"type": "raster", "width": W}`; `width` is the default for contigs
`contigs`	yes	list of contig entries (9.3), in page order
`planes`	yes if extension planes are present	plane descriptions (9.2)
`reference`	RECOMMENDED	`{"name": "GRCh38", "source": "<file name>", ...}`
`sample`	yes if any `gt` plane is present, else `null`	sample description (9.4)
`palette`	no	name of the palette used for the `ref` colour maps
`generator`, `created`	no	software and ISO 8601 creation time
`summary`	no	the writer's totals for the conversion (informative)

Key	Required	Content
`subset`	yes, with `gpxil`, if the file has a later version than the version whose rules made its planes (below); else no	for a file that copies contigs from another without converting calls again: `{"of": "<that file's name>", "gpxil": "<the version whose section 6 its planes follow>", "contigs": [...], ...}`
`contig`, `plane`, `length`	yes	page 0's own page metadata (8.4)

A file's version is the version whose rules it follows, including those of section 6 for its genotypes. A file that copies planes from another without converting calls again (a working copy of some contigs, or windows cut out of them) keeps that file's version, since its planes were made by that version's rules. But a file that holds a window (section 3.1) has version 0.3 or later, the first with windows; if it copies a file of an older version, its `subset` key gives that version, whose section 6 its genotypes follow, and a copy of such a copy gives the version its source's `subset` key gives. A reader of version 0.2 or earlier reads a window as an ordinary contig, with positions counted from its first base.

9.2 Planes

`planes` maps plane names to objects with `dtype` (`"uint8"`, `"uint16"`, `"uint32"`, `"int8"`, `"int16"`, `"int32"` or `"float32"`) and optionally `missing` (the value meaning not available), `description` and `units`. Extension planes MUST be described here.

9.3 Contig entries

Key	Required	Content
`name`	yes	contig name: printable ASCII (0x21-0x7E), so no spaces, tabs or other whitespace; unique in the file; SHOULD follow the SAM naming rules
`length`	yes	`L`
`pages`	yes	map from plane name to page index

Key	Required	Content
`width`	no	layout width `W` of this contig, if different from `layout.width`
`md5`	RECOMMENDED	MD5 of the sequence (below), as in SAM `@SQ M5`
`refget`	RECOMMENDED	GA4GH refget identifier `SQ.<sha512t24u>` of the sequence (below)
`aliases`	no	other names of this contig (e.g. `"21"` for `"chr21"`)
`ploidy`	no	from the `gt` plane: `1` if haplotype 2 is ABSENT at every base, `2` if at none, else `"mixed"`
`vcf_contig`	no	the VCF contig name that was encoded
`encoding`	no	conversion counts (9.4)
`callable_bases`	no	number of bases inside callable regions
`sidecar_entries`	no	number of sidecar entries
`phase_sets`	no	number of distinct non-zero values in the `ps` plane
`phase_set_ids`	yes if `ps` holds replaced identifiers	map from `ps` values (as decimal strings) to the original `FORMAT/PS` identifiers they replace (5.5)
`invalid_characters_converted_to_N`	yes if any	number of FASTA characters stored as N (5.1)
`window`	yes for a window (3.1)	where in its parent the window lies (below)

Both digests are computed over the sequence as decoded from the `ref` plane in uppercase: one IUPAC letter per base and `-` for "no base", without line breaks. For a FASTA of IUPAC letters this equals the digest other tools compute from the FASTA; for one that used `.` or `U`, or had characters converted to N, it does not.

A window's `window` key is an object:

```
{
  "of": "chr17",
  "start": 7561778,
  "length": 83257441,
  "md5": "a8499ca51d6fb77332c2d242923994eb",
  "refget": "SQ.upqChCoU-Gtd_61IidCsln-r8cxUTFeP",
  "aliases": ["17"]
}
```

`of` (REQUIRED) is the parent's name; `start` (REQUIRED) the offset in the parent of the window's offset 0, a whole number; `length` (RECOMMENDED) the parent's length; `md5` and `refget` (RECOMMENDED) the parent's digests; and `aliases` (RECOMMENDED when the parent has other names) the parent's other names, by which readers can find it.

In a window's entry, `md5`, `refget`, `ploidy`, `callable_bases`, `sidecar_entries`, `phase_sets` and `phase_set_ids` describe the window; `aliases` are other names of the window, never names of its parent (those are `window.aliases`); `vcf_contig` and `invalid_characters_converted_to_N` describe the parent; and `encoding` counts what converting calls for the window did, which can include calls next to it (section 3.1). A window cut from another file without converting calls again SHOULD leave out `encoding` and `callable_bases`, which it cannot know.

9.4 Sample and conversion record

`sample` is `null` for a file without `gt` planes, including one made from a VCF none of whose calls fell on a contig of the reference; otherwise it is an object with `id` (REQUIRED, the VCF sample name) and, as applicable, `source` (input file name), `callable_regions` (the name of the

callable-regions file applied in section 6.1, or `null`), `haploid_regions` (a description of the haploid regions applied, such as a file name, a list of whole contigs or the rule used for a sex, or `null`) and `sex`.

Each contig's `encoding` object SHOULD report the counts named in section 6: `records`, `conflicts` (6.4), `ref_mismatches` (6.7), `skipped_out_of_range` and `clipped_to_contig_end` (6.2), `bad_allele_index` (6.2), `symbolic_alleles` (6.3), `ploidy_mismatches` (6.4), `missing_genotypes`, `polyploid_skipped`, `spanning_deletion_alleles`, `spanning_deletions_reassigned` and `overlapping_calls_reassigned` (6.5), and `nonacgt_inserted_bases` (7).

10. Conformance

A **conforming writer** produces files in which:

1. the TIFF structure, page order, tags and page metadata follow section 8;
2. every plane has the dimensions of section 4 and zero padding;
3. every code is valid (section 5, Appendix A): `ref` bits 5-7 clear and no 0x10; `gt` states from the table; reserved `flags` bits clear; a base state never equals the reference base; `ps` is 0 wherever PHASED is clear;
4. the planes of each contig meet the rules of section 3, and windows those of section 3.1;
5. the sidecar follows section 7: sorted, INS flags and `INS` entries one to one, every COMPLEX run starting with one `COMPLEX` entry;
6. the metadata contains the required keys (section 9), including a `sample` object when any `gt` plane is present, and, where present, digests that match the `ref` plane;
7. calls from a VCF are converted as in section 6, and a window holds what its whole parent would (section 3.1).

A **conforming reader**:

1. reads the metadata from page 0 and finds pages through `contigs[].pages`;
2. decodes uncompressed and Deflate pages, in strips or tiles;
3. ignores padding, unknown metadata keys, unknown extension planes, unlisted pages and unknown sidecar kinds;

4. treats reserved `gt` states as UNKNOWN;
5. does not use colour maps to interpret data;
6. reads files of any version 0.x, and rejects files of another major version (section 14);
7. gives positions on a window in its parent's coordinates (section 3.1), or says that they are counted from the window's first base.

`gpxil validate` in gpxiltools checks a file against items 1-6 of the writer list (item 7 needs the original VCF; the test suite of gpxiltools covers it, and checks that windows converted on their own and cut out of whole contigs agree). Of item 7 it checks what a file shows: that a window holds what the file's whole parent holds over its stretch, and that windows of one parent agree where they overlap.

11. Interpretation (informative)

11.1 Haplotype sequence

The sequence of haplotype `h` over offsets `[s, e)` is built base by base: REF gives the reference base; A, C, G or T gives that base; UNKNOWN and COMPLEX give N; DEL and ABSENT give nothing; after each base with INSh set, the sidecar's inserted bases follow. Soft-mask case can be carried over from the reference (the convention of `bcftools consensus`).

11.2 Genotype as one code

The union of the two haplotype bases at a position (`haplotype 1 base | haplotype 2 base`, with DEL and ABSENT contributing nothing and UNKNOWN or COMPLEX contributing N) is an IUPAC code: a heterozygous A/G site is R. This is the natural single-code picture of a diploid genome, and what rendering uses for heterozygous colours (section 12.3).

11.3 Dosage

Counting, at every position, how many haplotypes carry A, C, G and T gives an `(L, 4)` array of allele dosages: the one-hot encoding used by DNA sequence models, multiplied by ploidy. It is computed from `ref`

and `gt` and is not stored; storing it instead would lose phase, insertions and the distinction between deleted and unknown bases (see docs/rationale.md).

11.4 Back to VCF

A gpxil file converts back to VCF by emitting one record per *event stretch*: each substitution is a one-base stretch; a run of deleted bases takes the anchor base before it; an insertion takes its anchor base; stretches that share a base are merged. REF is the reference over the stretch and each haplotype contributes its sequence over the stretch as an allele. The genotype is written phased only if every called base of the stretch is PHASED. Runs of UNKNOWN outside these stretches can be written as gVCF-style blocks (`ALT=<*>`, `INFO/END`, `GT=./.`) so that re-encoding restores them. Re-encoding such an export gives the same haplotype sequences; individual `gt` values can differ where an indel inside a merged stretch is placed at a different but equivalent position.

The records of a window (section 3.1) are its parent's over its stretch: `CHROM` is the parent's name and offset `i` is `POS = start + i + 1`, while `END` in COMPLEX data is already the parent's and `ps` values are kept. At its ends a record says only what the window holds: an event that the window cuts is written as far as the window holds it. A deletion at the window's first base has no anchor inside the window, so the record takes the base before the window as its anchor, at `POS = start`, written `N` (the window does not hold it), and covers only the deleted bases the window holds. (A contig of a whole sequence never starts with a deleted base, since section 6 deletes only bases after an anchor; so a window that starts inside a deletion has `start` > 0.)

A file's windows of one parent that overlap or touch, and the parent if the file holds it, are read as one stretch of the parent: their records are those of a window of all of that stretch, each once, and a region of the parent that the stretch holds has the records that such a window gives for it. Where they have different planes, the stretch has a plane where any of them has it, and where none that has it holds a base, the base is unknown there (UNKNOWN on both haplotypes, no flags, quality, depth or phase set); writers of VCF leave out what windows without the sample's planes would add, as they leave out contigs without a `gt` plane.

11.5 Image analysis

Because planes are aligned to the reference, image operations are genome operations:

- **comparison**: the pixel-wise difference of two people's `gt` planes lists where their genomes differ;
- **cohorts**: the mean of many dosage images is an allele-frequency image;
- **density**: a box filter over the variant mask of `gt` is a variant-density map, and over the G/C bits of `ref` a GC-content (isochore) map;
- **periodicity**: re-wrapping `ref` to a width equal to a repeat period turns tandem repeats into vertical stripes, which Albrecht-Buehler used to find tetra-periodic motifs;
- **two-dimensional locality**: a Hilbert-curve view keeps runs of bases together in 2-D blocks (a `2^k × 2^k` block is `4^k` consecutive bases), so convolutional filters see genomic neighbourhoods.

12. Rendering and printing (informative)

12.1 Data are not colours

A pixel value is a code; a palette turns codes into colours when the image is displayed or printed. This separation is what lets one file serve both the screen (sRGB) and the press (CMYK). Converting colour pixel data between colour spaces is lossy: CMYK has four channels and RGB three, so many CMYK values print the same colour, and ICC conversions apply gamut mapping and black generation. Data stored as colours do not survive such conversions. Instead, a print rendering maps codes directly to ink values chosen in CMYK, and a screen rendering maps the same codes directly to sRGB.

12.2 Palettes

A palette gives the four bases, N and "no base" in both sRGB and CMYK (ink coverage in percent); other codes are derived:

- **ambiguity codes and heterozygous sites** mix their component bases, either as the mean of the colours, or by *overprinting* (every component ink at full strength, multiplying transmittance on screen);

- **soft-masked bases** are faded towards paper white by a fixed fraction. N and "no base" are not faded.

N (not known) has a colour unlike every base and every mixture, faded or not, so that an unknown base is never mistaken for a heterozygous site. The `classic` and `okabe-ito` palettes draw it near-black, because their mixtures include greys: in `classic`, C (blue) and T (yellow) average to mid-grey.

Palette	A	C	G	T	Mix	Notes
`classic`	red	blue	green	yellow	mean	colours of the original gpxil notebooks
`gpxi`	black	dark grey	white	light grey	mean	Albrecht-Buehler (2009) grey levels; no fade
`okabe-ito`	bluish green	blue	orange	vermillion	mean	safe for colour-blind readers; IGV-like order
`process`	cyan	magenta	yellow	black	overprint	one process ink per base

Exact values are in Appendix A.4. Under the `process` palette the 4-bit IUPAC code *is* a CMYK pixel at one bit per ink: bit A lays down cyan, C magenta, G yellow, T black. A heterozygous A/G site prints cyan and yellow; four channels for four bases, with each code mapping to exactly one ink combination.

12.3 Render modes

- **reference**: the `ref` plane.
- **genotype**: the union code of section 11.2, so homozygous bases show their base colour, heterozygous sites the mixture, and bases deleted on both haplotypes show as paper.
- **haplotypes**: two pixel rows per row of bases, haplotype 1 above haplotype 2. Lossless for substitutions and deletions.
- **variants**: the reference faded, with variant sites coloured by zygosity (heterozygous, homozygous, deletion, insertion, no call).

12.4 A genome book

Printed at one dot per base on A4 paper at 300 dots per inch, with 15 mm margins and room for a one-line running header and footer (as gpxiltools lays out its books), a page holds 2,970 rows of 2,126 bases: 6,314,220 bases. Starting each chromosome on a new page, the 25 sequences of GRCh38's chromosomes 1-22, X, Y and the mitochondrion (3.09 billion bases) fill 505 pages; at 600 dots per inch they fill 137. Each page is one indexed image whose colour table is in DeviceCMYK, with interpolation off so every base stays a square dot. Colours given in device CMYK should be proofed with the printer's ICC profile.

13. Privacy and security considerations

A gpxil file with a `gt` plane is a person's genome, and so is every faithful rendering of it. A genome book printed at one dot per base is the genome in printed form: scanned at sufficient resolution it can be read back. Treat files, renderings and books of individuals with the same care as the VCF or CRAM they came from: they are identifying, and they reveal information about relatives. Pictures intended for display or sharing should use a reference-only or deliberately reduced rendering. Metadata may contain sample identifiers and file names.

Readers parse JSON, a zlib stream and TIFF structures from untrusted input and should bound memory use (a page header can claim a very large image) and reject inconsistent sizes.

14. Versioning and future work

The version is `major.minor`. Minor versions add optional features (new planes, metadata keys, sidecar kinds, state codes taken from the reserved values) that older readers can ignore safely, or refine the conversion of VCF calls (section 6), which changes what writers produce but not what readers read; a file says which version's rules made it (section 9.1). A new major version is not readable by older readers. Readers MUST reject files with a major version they do not implement.

Planned or possible extensions:

- multi-sample (cohort) files, as stacks of `gt` planes, and a Zarr binding for analysis at cohort scale;
- reduced-resolution overview images (TIFF SubIFDs) for zooming viewers;
- a standard plane for base modifications (e.g. 5mC methylation from long-read sequencing);
- an OME-TIFF compatible profile for bioimaging software;
- polyploid genotypes and richer structural-variant support;
- preview pages for ordinary viewers.

While the format is a draft its versions may still be clarified. Changes so far:

- **28 October 2026, version 0.3**: windows (section 3.1). A contig can hold a stretch of a longer sequence, its parent, and say where in it the stretch lies, so that a gene and its surroundings travel in a file of a few kilobytes and keep the chromosome's positions. A window holds what a contig of its whole parent would hold over its stretch, whether its calls were converted on their own or it was cut out of a whole contig; its name differs from its parent's, and is best the region it holds (`chr17:7561779-7787546`); readers give positions on it in its parent's coordinates, and a reader that matches names loosely matches only whole contigs so. New: the `window` key of contig entries (9.3), the `subset` key (9.1), the version `"0.3"` and its schema. Section 11.4 now calls the stretches that VCF records cover *event stretches*, leaving "window" to windows, and says how windows' records are written: a deletion at a window's first base is anchored on the base before it, written `N`, and windows that overlap or touch are read as one stretch. Phase-set identifiers that are not integers are now numbered counting every phased call's identifier, including one that ends up on no base, and MUST be (5.5), so that a window's values are its parent's; an integer identifier is one written in the digits 0-9. And 6.2 now states that a `<DEL>` allele's `SVLEN` extends a span as its `END` would. Codes, planes, the sidecar and the container are unchanged, so readers of 0.2 read files of 0.3 (a window as an ordinary contig), and files of 0.1 and 0.2 remain valid.
- **28 October 2026, version 0.2**: unphased calls that overlap other calls are put on the haplotype they fit (section 6.5). Two heterozygous calls that overlap cannot be on one haplotype, yet version 0.1 applied both as written, to haplotype 2, where they ran

together: two deletions became one, and an insertion followed a base that another call deletes. Split multi-allelic records, unphased, now go one to each haplotype, unless their alleles change different bases. Comparing HG002's two GIAB benchmarks, and HG002 with his parents, base by base found the problem; on chromosome 17 of the three v4.2.1 benchmarks the rule moves 26 of 320,884 records to the other haplotype (on their 22 autosomes, 687 of 12,079,785), and the comparisons then agree with a record-by-record check everywhere. New: the count `overlapping_calls_reassigned` (9.4), the version `"0.2"` (9.1) and its schema. Made precise: an allele number is written in the digits 0-9, and a record with an empty REF is skipped (6.2); two insertions differ if the sidecar stores different bases, two COMPLEX alleles if their REF, ALT or END differ (6.4). Codes, planes, the sidecar and the container are unchanged, so readers of 0.1 read files of 0.2, and files of 0.1 remain valid files of version 0.1.

Version 0.1:

- **26 October 2026**: first draft.
- **27 October 2026**: after a review against the reference implementation, the VCF conversion was made precise where writers could have disagreed (weak and strong states, missing genotypes, unphased `*` alleles, symbolic alleles and `SVLEN`, records past the contig end, contigs without calls, IUPAC REF alleles, annotation precedence), and the sidecar, metadata and conformance rules were tightened (COMPLEX entries, `ps` without PHASED, `sample` required with genotypes, printable ASCII names, digests of the decoded sequence). Codes, planes and the container are unchanged; files written by gpxiltools 0.1.0 remain readable.
- **28 October 2026**: N became near-black in the `classic` and `okabe-ito` palettes and is no longer faded for soft-masking, so that it differs from every base and mixture (12.2, A.4). In `classic`, a mid-grey N was exactly the colour of a heterozygous C/T site. Palettes are rendering choices: files are unchanged.

15. References

- Albrecht-Buehler G (2009). Outline of a genome navigation system based on the properties of GA-sequences and their flanks. *PLoS ONE* 4(3): e4701. https://doi.org/10.1371/journal.pone.0004701
- Adobe (1992). *TIFF Revision 6.0*; and the BigTIFF extension (2007).
- SAMtools / hts-specs. *The Variant Call Format Specification*, v4.3 and v4.4.
- SAMtools / hts-specs. *Sequence Alignment/Map Format Specification* (BAM 4-bit base encoding).
- GA4GH. *refget* sequence retrieval specification (sha512t24u identifiers).
- Cornish-Bowden A (1985). Nomenclature for incompletely specified bases in nucleic acid sequences (IUPAC-IUB). *Nucleic Acids Research* 13: 3021-3030.
- Okabe M, Ito K (2002). *Color Universal Design*.
- Wagner J et al. Genome in a Bottle HG002 benchmark sets (used for the measurements in docs/rationale.md).

Appendix A: code tables

A.1 `ref` low nibble

Code	Symbol	Bases		Code	Symbol	Bases
0x0	`-`	none		0x8	T	T
0x1	A	A		0x9	W	A T
0x2	C	C		0xA	Y	C T
0x3	M	A C		0xB	H	A C T
0x4	G	G		0xC	K	G T
0x5	R	A G		0xD	D	A G T
0x6	S	C G		0xE	B	C G T
0x7	V	A C G		0xF	N	A C G T

Add 0x10 for a soft-masked (lowercase) base. Valid `ref` bytes are 0x00-0x0F and 0x11-0x1F.

A.2 `gt` nibble states

Nibble	0	1	2	3	4	5	6	7	8	9	A	B	C	D
State	REF	A	C	–	G	–	–	–	T	–	ABSENT	–	COMPLEX	DEL

(– = reserved)

A.3 `flags` bits

`0x01` PHASED, `0x02` INS1, `0x04` INS2, `0x08` FILTERED, `0x10` VARIANT_SITE, `0x20` `0x40` `0x80` reserved.

A.4 Palette values

sRGB as 0-255 triples; CMYK as ink percentages.

Palette	Base	sRGB	CMYK %
classic	A	255 0 0	0 100 100 0
	C	0 0 255	100 80 0 0
	G	0 255 0	80 0 100 0
	T	255 255 0	0 0 100 0
	N	32 32 32	0 0 0 90
gpxi	A	0 0 0	0 0 0 100
	C	85 85 85	0 0 0 67
	G	255 255 255	0 0 0 0
	T	170 170 170	0 0 0 33
	N	170 200 235	30 10 0 0
okabe-ito	A	0 158 115	97 0 75 0
	C	0 114 178	100 50 0 0
	G	230 159 0	0 50 100 0
	T	213 94 0	0 80 100 0
	N	32 32 32	0 0 0 90
process	A	0 174 239	100 0 0 0
	C	236 0 140	0 100 0 0
	G	255 242 0	0 0 100 0
	T	35 31 32	0 0 0 100
	N	160 160 160	0 0 0 35

"No base" is paper: sRGB 255 255 255, no ink (the `gpxi` palette tints it 255 250 230 / 0 2 10 0 so that it differs from G). Soft-masked bases are faded 45 % towards paper in sRGB and their ink reduced by 45 % in CMYK (`gpxi`: no fade); N and "no base" are never faded. sRGB values of the `process` palette are screen approximations of the inks.

Appendix B: worked example

Reference `chrT` = `ACGTACGTac` (10 bases, the last two soft-masked), layout width 4, and one sample with these calls:

```
#CHROM POS REF  ALT   FILTER GT
chrT   2   C    T     PASS   0|1
chrT   4   TAC  T     PASS   1|0
chrT   8   T    TGG   PASS   1|1
chrT   9   A    G     LowQ   ./1
```

Planes (offsets 0-9; padding offsets 10 and 11 are 0):

offset	base	`ref`	`gt`	meaning	`flags`
0	A	0x01	0x00	REF|REF	0x00
1	C	0x02	0x08	REF|T	0x11 PHASED, VARIANT_SITE
2	G	0x04	0x00		0x00
3	T	0x08	0x00	anchor of the deletion	0x11
4	A	0x01	0xD0	DEL|REF	0x11
5	C	0x02	0xD0	DEL|REF	0x11
6	G	0x04	0x00		0x00
7	T	0x08	0x00	insertion follows on both	0x17 PHASED, INS1, INS2, VARIANT_SITE
8	a	0x11	0xF4	UNKNOWN|G	0x18 FILTERED, VARIANT_SITE
9	c	0x12	0x00		0x00

Sidecar:

```
#gpxil-sidecar  0.1
offset  hap kind    data
7   1   INS GG
7   2   INS GG
```

Following section 11.1, haplotype 1 is `A C G T` (offsets 4 and 5 deleted) `G T`, the inserted `GG`, then `N` (unknown) and `C`: `ACGTGTGGNC`. Haplotype 2 is `ATGTACGTGGGC`. Carrying the reference's soft-mask case over, they read `ACGTGTGGnc` and `ATGTACGTGGgc`.

As images with `W = 4`, each plane is 3 rows of 4 pixels; the `ref` page shows `ACGT / ACGT / ac` followed by two padding pixels.

The file `examples/worked-example.gpxil.tif` in this repository contains exactly this example.

A window of the example (section 3.1). The window `chrT:5-10` holds offsets 4 to 9 of `chrT`, positions 5 to 10. Its metadata entry has `"window": {"of": "chrT", "start": 4, "length": 10, ...}`, with `chrT`'s digests, and its planes are those of the table above from offset 4 on: `ref 01 02 04 08 11 12`, `gt D0 D0 00 00 F4 00` and `flags 11 11 00 17 18 00`, laid out from row 0, column 0 of its own pages. The insertion's two sidecar entries are at the window's offset 3. Converted on its own, the window takes the record at POS 4, which deletes two of its bases from an anchor outside it, and those at 8 and 9; not the one at 2, which overlaps none of them. Haplotype 1 over the window is `GTGGnc` and haplotype 2 `ACGTGGgc`. Back to VCF (section 11.4), the deletion has no anchor in the window, so it takes the base before the window, which the window doesn't hold, as `N`:

```
#CHROM POS REF ALT FILTER         GT
chrT   4   NAC N   PASS           1|0
chrT   8   T   TGG PASS           1|1
chrT   9   A   G   gpxil_filtered ./1
```

The file `examples/worked-example-window.gpxil.tif` holds this window, and `examples/worked-example-window.expected.json` what a reader finds in it.